

Unix and Bash exercises

NUnix · L3 MIAGE · TD

ISTIC · University of Rennes 2026–2027



1/13

Exercise session



- 1. Filename patterns
- 2. Redirections and paths
- 3. Scripts with arguments and interactive input
- 4. Search commands

Use a **practice directory** for files that you create or change.
Read commands and predict their effects before executing them.

2/13

Exercise 1: filename patterns



List the entries in /usr/bin whose names:

- 1. Start with m.
- 2. Start with m and have exactly three characters.
- 3. Start with m and have at least three characters.
- 4. Start with m and have an extension after a dot.

3/13

Exercise 2: the initial files



Directory	Contains
practice/user1/	f1, f2, f3
practice/user2/	f4

The current directory is **practice/user2/**.
What relative path names user1's file f1?

4/13

Exercise 2: redirections with cat



Using cat and redirections:

- 1. Create f5 containing the line 5 2 3 -1.
- 2. Create f6 from f4, followed by f5.
- 3. Append user1's f1 to f6.
- 4. Copy user1's f2 into a new file f7 in user2.

5/13

Exercise 3: a script with three filenames



Write a script named execution, called with n1 n2 n3.

Step	Input	Program	Output
1	File n1	p1	File n1. r1
2	File n2	p1	File n2. r1
3	n2. r1, then n1. r1	p2	File n3

p1 and p2 read **stdin** and write **stdout**.
Concatenate the two intermediate files in the stated order.

6/13

Exercise 3: resolving the paths



Directory	Contains
practice/user1/	file1, file2, p1, p2, execution
practice/user2/	Another file named file2

From practice/user1/, run:

```
./execution file1 ../user2/file2 fileN
```

Which files are read? Where do the three output files appear?

7/13

Exercise 3: interactive input



Write another version of execution that asks the user for:

- ▶ The first input filename.
- ▶ The second input filename.
- ▶ The output filename.

Keep the same processing steps and concatenation order.

8/13

Exercise 4: script arguments



Write argument, using expr to update a counter.

```
./argument one two three
```

Expected output:

```
3 arguments:
argument 1 - one
argument 2 - two
argument 3 - three
```

9/13

Exercise 5: finding a pattern



Write mysearch, called with a pattern and a filename:

```
./mysearch 'error' application.log
```

Print either:

- ▶ application.log has the pattern
- ▶ application.log does not have the pattern

Handle an unreadable or missing file as an **error**.

10/13

Exercise 5: repeated searches



1. Write mysearch2 filename. Ask for a pattern repeatedly until the user enters **end**.
2. Write mysearch3 f1 ... fn. Search the supplied files for each entered pattern.

Where does each loop belong? What happens when input reaches EOF?

11/13

Exercise 6: reading find commands



Explain each command. Identify unnecessary directory visits or ambiguous patterns.

```
find /var/log -exec grep 'www.athabasca.com' {} \;
find . -name 'rc.conf' -exec chmod o+r {} \;
find /usr/local -type f -name '*.html'
```

12/13

Exercise 6: who handles the redirection?



Analyse this command before running it:

```
find . -name 'tp2' -exec {} < f_in > {}.out \;
```

1. Which program opens `f_in` and `{}.out`?
2. Does `find` replace `{}` inside the redirection filename?
3. How would you give each matching program its own output file?

13/13